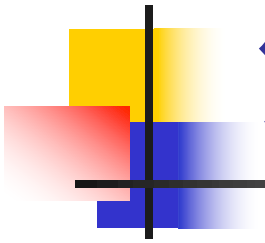


言語処理系 第5回
LR構文解析(2)・内部表現基礎

木村啓二
keiji@waseda.jp

中間試験

- 6月5日授業時間中に教室で実施
  - 試験時間は1時間
  - 答案回収後、残り時間は解説
- 資料参照OK・相談NG
  - 解に至った理由説明を重視

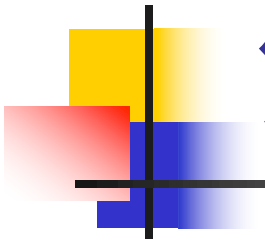


第4回演習問題(1)

- 文法Gのclosure({F→(.E)})を求めよ
- GOTO(I4, ()を求めよ
- GOTO(I4,i)を求めよ

第4回演習問題(1)解答例

- closure({F→(.E)})
  ={F→(.E), E→.E+T, E→.T, T→.T*F,
  T→.F, F→.(E), F→.i}
- GOTO(I4,()
  = closure({F→(.E)}) （以下略）=I4
- GOTO(I4,i)
  = closure({F→i.}) = {F→i.#}=I5



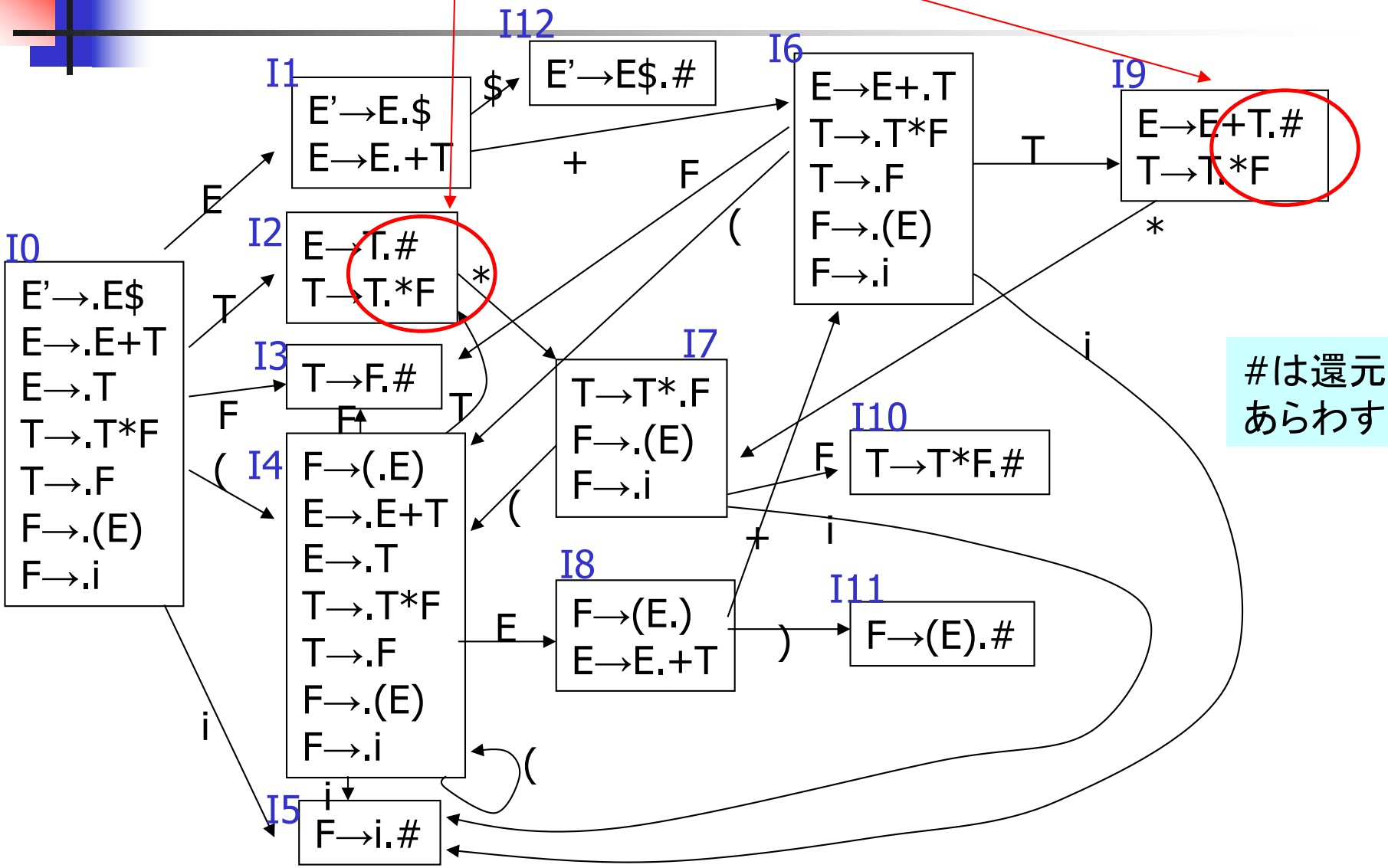
第4回演習問題(2)

- 文法Gに関してSLR(1)構文解析表を求める
  - A[I9, +], A[I9,)], A[I9, $], A[I9, *]をそれぞれ求めよ
    - ヒント: どの記号のFollow集合を使うか？
  - I9で還元となるLR(0)項はE→E+T.#なので、Follow(E)を考える。文法GではFollow(E)={+,),$}なので、
    A[I9,+]=A[I9,)]=A[I9,$]={E→E+Tで還元}
    A[I9,*]=s7

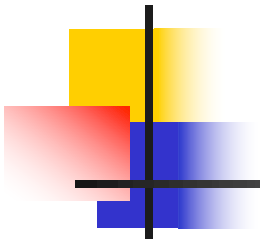
とりあえず先読みして
シフト・還元を決定する（ものとする）

シフト・還元の曖昧性が生じてしまい，問題となる

文法Gの状態遷移図



#は還元をあらわす



LR(k)文法

- 左から右へトークンをk個先読みしつつ最右導出できる文法のクラス
  - LL(k)文法より表現できる範囲が広い
- 上向き構文解析で解析器を実現
- 今回は以下を取り扱う
  - SLR(1)
  - **LR(1)**
  - **LALR(1)**
- 解析器の自動生成ツールがある

シフト・還元競合をどう回避するか？
→先読みトークンとそれによる
　遷移先の状態をどう関連付けるか？

LR(1)構文解析

- SLR(1)で導入したFollow集合は過去を覚えていない

$$Follow(A) = \{a | a \in V_T, S \Rightarrow^* \cdots Aa \cdots\}$$

- 過去も考慮すると文法の範囲が広がる
  - LR(1)項
    - [A→x.y,z]: 生成規則の直後に来る一つの終端記号を付加
      - A→xyをxまで読み進んで、このあとyまで読むと次にzが来る
    - 「.」が右端にあるLR(1)項は指定した終端記号で還元することを表す
    - [A→xy.,z]なら、先読みでzを読んだらAに還元する
  - LR(1)項の集合Iに対する閉包closure(I)
    - 1. closure(I)=I
    - 2. $[A \rightarrow \alpha . B \beta , a] \in closure(I)$に対して$B \rightarrow \gamma$なる生成規則があればb∈First(βa)なる全てのbについて$[B \rightarrow . \gamma , b]$をclosure(I)に追加 (b∈Follow(B)であることに注意)
    - 3. 2.を新たに追加するものがなくなるまで繰り返す

Aのaによる還元に至る終端記号を求めていく

  - LR(1)項を用いたGOTO(I,X)
    - $GOTO(I, X) = closure(\{[A \rightarrow \alpha X . \beta , a] | [A \rightarrow \alpha . X \beta , a] \in I\})$
  - 競合の解決にLR(1)項を用いる
  - LR(0)項では同一の状態が，追加された終端記号（還元に至る記号）で別々の状態になる→状態遷移表が巨大に

LR(1)項の集合Iに対する閉包 closure(I)に関する補足

- 前ページ「[A→α.Bβ,a]∈closure(I)に対して B→γなる生成規則があればb∈First(βa)なる全てのbについて[B→.γ,b]をclosure(I)に追加」
  - βがεならb∈First(βa)はすなわちb∈First(a)={a}なので[B→.γ,a]
  - つまり[A→α.B,a]∈closure(I)に対して B→γなる生成規則があれば[B→.γ,a]をclosure(I)に追加
    - Aの導出課程において，途中のBからの導出過程を考慮できる
    - LR(0)項の自然な拡張
  - A→α.BβのLR(1)項は…
    - Aの後に来る記号を考慮できる
    - Aを構成するBの後に来る記号（Aの途中経過）を考慮できる
  - 考慮しすぎなので状態も多くなる

例：closure({E′→.E,$})

- E′→.E$においてEの後は$（Eと$の間は何もない）なので、[E→.E+T,$], [E→.T,$]を追加
  - [E→.T,$]から芋づる式に[T→.T*F,$], [T→.F,$]を追加。同様に[F→.(E),$], [F→.i,$]も追加
- [E→.E+T,$]に対して".E"の後のFirst(+T$)={+}なので、[E→.E+T,+], [E→.T,+]を追加
  - 先と同様に[T→.T*F,+], [T→.F,+], [F→.(E),+], [F→.i,+]も追加
  - [E→.E+T,$]と[E→.E+T,+]は[E→.E+T,$/+]とまとめて表記する
- [T→.T*F,$]に対してFirst(*F$)={*}なので、[T→.T*F,*], [T→.F,*]を追加
  - 先と同様に, [F→.(E),*], [F→.i,*]も追加

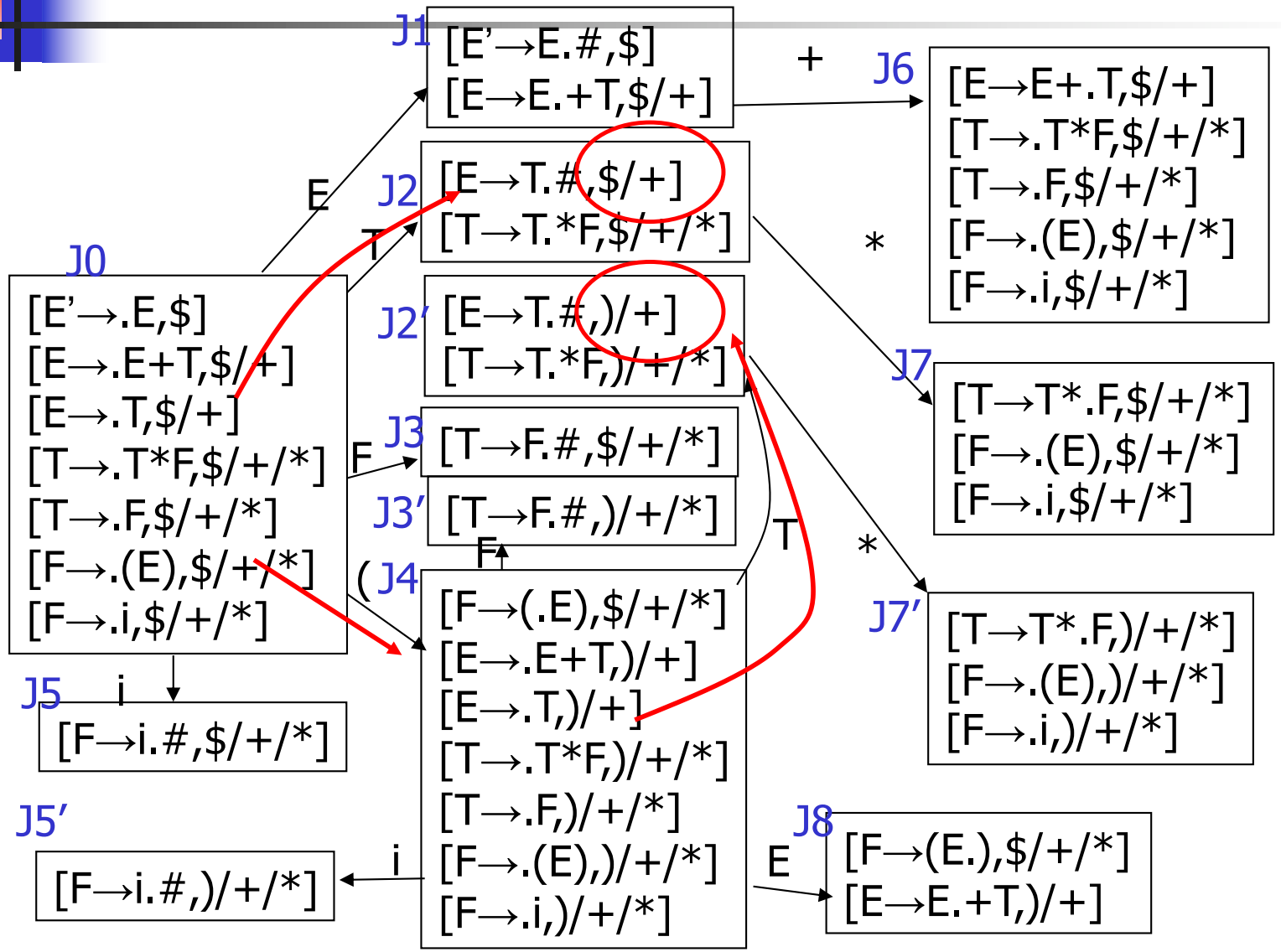
LR(1)構文解析表（一部）

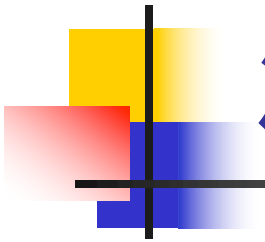
- 先と同じ文法G（状態をJnとして表記）
  - J0=closure({E′→.E,$})=
    {[E′→.E, $]
    [E→.E+T,$/+]
    [E→.T, $/+]
    [T→.T*F, $/+/*]
    [T→.F, $/+/*]
    [F→.(E), $/+/*]
    [F→.i, $/+/*]}
  - J2=GOTO(J0, T) =
    [E→T.#, $/+]
    [T→T.*F, $/+/*]}

[E→.E+T, $]からFirst(+T$)={+}なので、[E→.E+T,+]と[E→.T,+]も追加

先の状態遷移表（解析表）ではI2で**Tを読んだ時点**でシフトするか還元するか不明であったが、J2は先読みで$か+であれば還元することがわかる。

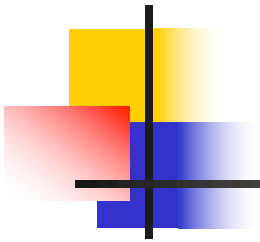
LR(1)の状態遷移図(一部)





演習問題(1)

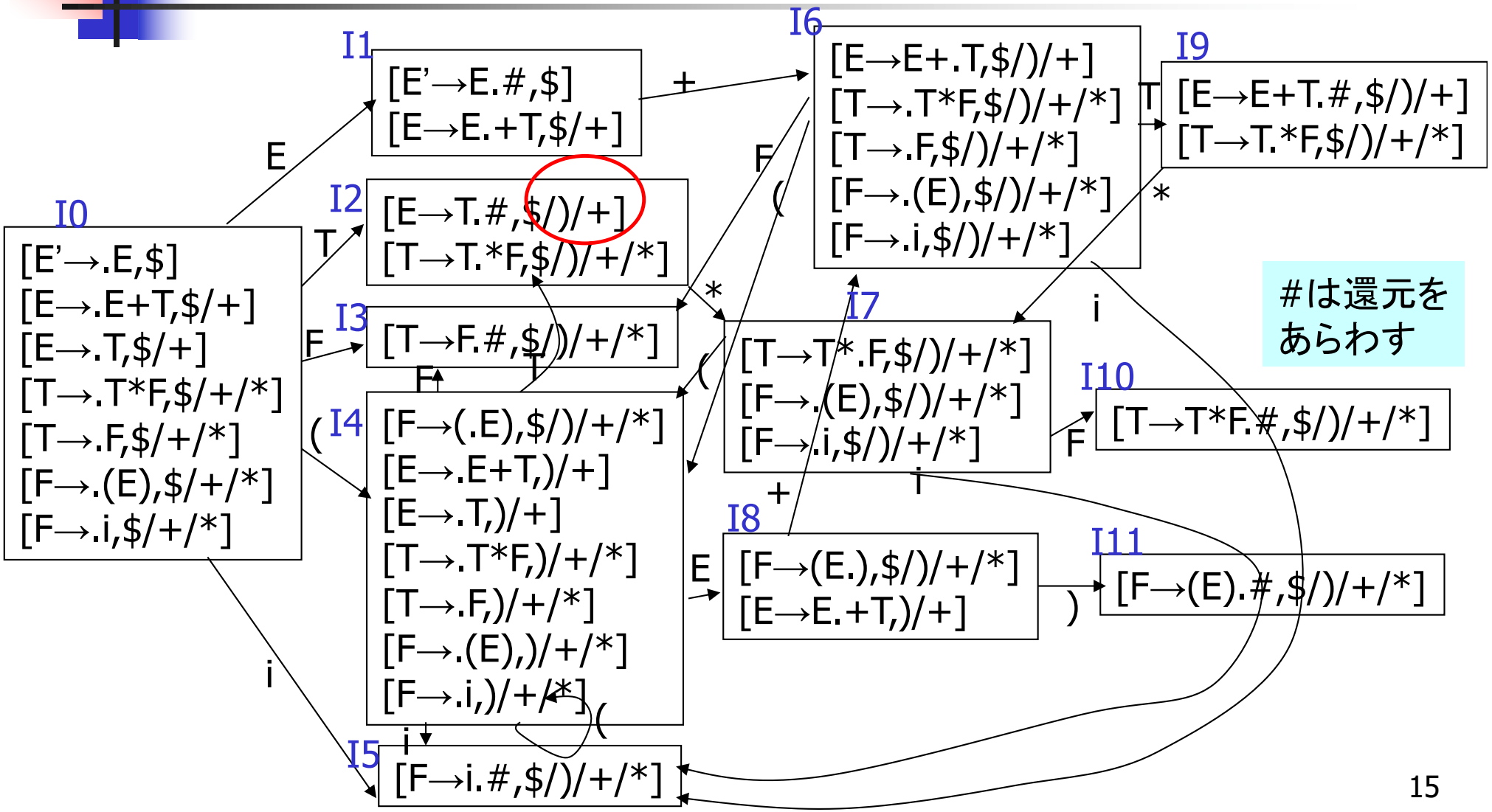
- 式"(i+j)"を先の「LR(1)の状態遷移図（一部）」で読み取ることを考える．"i"は状態遷移によりEまで還元されるが、Eに還元されるのはどの状態か？
- さらに上記の状態からどの記号の先読みによりシフトあるいは還元が決まるか？



LALR(1)構文解析

- 適用範囲がLR(1)とSLR(1)の中間
- LR(1)項の[A→x.y,a] のA→x.yを核と呼ぶ
- LALR(1)状態はLR(1)の状態の中で
  同じ核の集合を持つものを合併して作る

LALRの状態遷移図


I0
[E'→.E,$]
[E→.E+T,$/+]
[E→.T,$/+]
[T→.T*F,$/+/*]
[T→.F,$/+/*]
[F→.(E),$/+/*]
[F→.i,$/+/*]
I1
[E'→E.#,$]
[E→E.+T,$/+]
I2
[E→T.#,$/)/+]
[T→T.*F,$/)/+/*]
I3
[T→F.#,$/)/+/*]
I4
[F→(.E),$/)/+/*]
[E→.E+T,)/+]
[E→.T,)/+]
[T→.T*F,)/+/*]
[T→.F,)/+/*]
[F→.(E),)/+/*]
[F→.i,)/+/*]
I5
[F→i.#,$/)/+/*]
I6
[E→E+.T,$/)/+]
[T→.T*F,$/)/+/*]
[T→.F,$/)/+/*]
[F→.(E),$/)/+/*]
[F→.i,$/)/+/*]
I7
[T→T*.F,$/)/+/*]
[F→.(E),$/)/+/*]
[F→.i,$/)/+/*]
I8
[F→(E.),$/)/+/*]
[E→E.+T,)/+]
I9
[E→E+T.#,$/)/+]
[T→T.*F,$/)/+/*]
I10
[T→T*F.#,$/)/+/*]
I11
[F→(E).#,$/)/+/*]
E
T
F
(
i
+
*
)
#は還元をあらわす

yacc (bison)のサンプル LALR(1)の処理系

```
%{
#include  <stdio.h>
%}
%token  SYM_NONE
%token  SYM_ADD
%token  SYM_SUB
%token  SYM_MUL
%token  SYM_DIV
%token  SYM_CR
%token  SYM_INTEGER
%%

s : list
  ;

list :
     | list expr SYM_CR
       { printf("%d¥n", $2); }
     ;
```

```
expr : term
     | expr SYM_ADD term
       { $$ = $1+$3; }
     | expr SYM_SUB term
       { $$ = $1-$3; }
     ;

term : prim_expr
     | term SYM_MUL prim_expr
       { $$ = $1*$3; }
     | term SYM_DIV prim_expr
       { $$ = $1/$3; }
     ;
prim_expr : SYM_INTEGER
          ;
%%
#include "lex.yy.c"
/* remove enums and main() from
sample01.l
   flex sample01.l
   bison sample02.y
   gcc sample02.tab.c -o sample02 -ly -
ll */
```

$1はtermの値
$3はprim_exprの値
乗算結果は$$に代入され，
これは左辺のtermとして
利用可能

この動作時には
term（乗除算）は
処理済み

生成規則右辺
（SYM_CRは改行）

SYM_INTEGER（整数）は
prim_exprに還元
読み取った整数値は
prim_exprとして利用可能

還元時の動作($2は
2番目の記号exprに対応）

シフト・還元競合の際はシフトが優先される

Cの仕様書では？字句の規則

識別子はアルファベットか「_」ではじまり、任意の数の英数字か「_」の列である、という規則

- 識別子 (Identifier)

identifier:
identifier-nondigit
identifier identifier-nondigit
identifier digit
identifier-nondigit:
nondigit
universal-character-name
other implementation-defined characters

- 教科書的には正規表現で表される
`[_a-zA-Z][_a-zA-Z0-9]+`

Cの仕様書では？
構文の規則

式の構成要素を階層的に表している

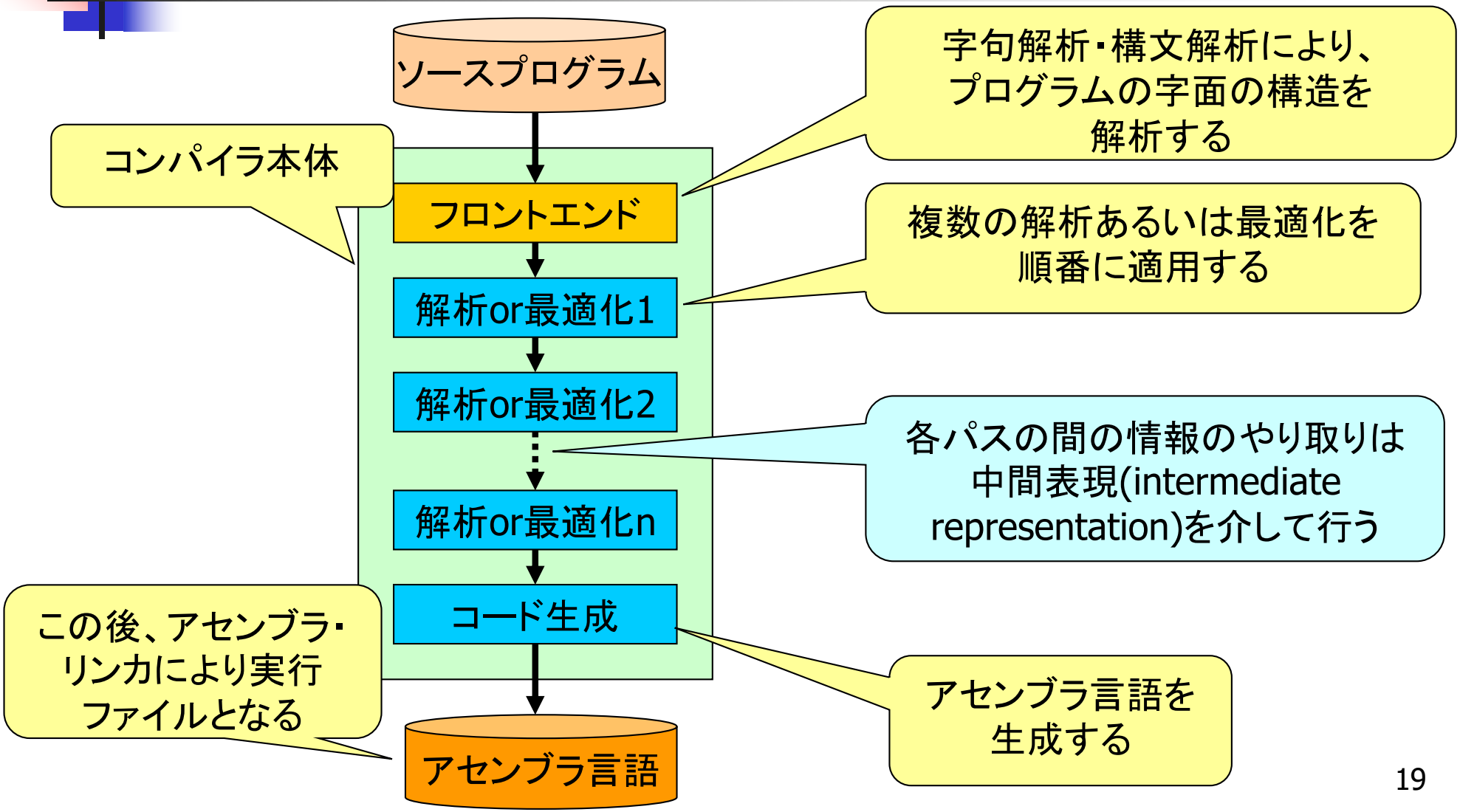
- 式（一部略）

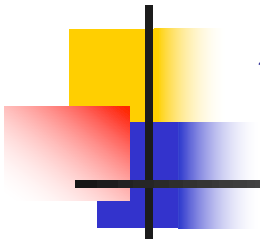
primary-expression:
　　identifier
　　constant
　　`(` *expression* `)`
postfix-expression:
　　postfix-expression `[` *expression* `]`
　　postfix-expression `++`
　　postfix-expression `--`
unary-expression:
　　postfix-expression
　　`++` *unary-expression*
　　`--` *unary-expression*
multiplicative-expression:
　　multiplicative-expression `*` *unary-expression*
　　multiplicative-expression `/` *unary-expression*
　　multiplicative-expression `%` *unary-expression*
additive-expression:
　　multiplicative-expression
　　additive-expression `+` *multiplicative-expression*
　　additive-expression `-` *multiplicative-expression*

- if文（一部略）

`if (` *expression* `)` *statement*
`if (` *expression* `)` *statement* `else` *statement*

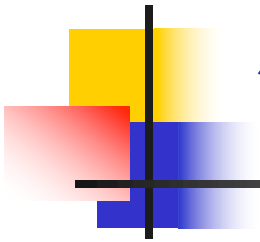
コンパイラの基本構成


ソースプログラム
コンパイラ本体
フロントエンド
解析or最適化1
解析or最適化2
解析or最適化n
コード生成
アセンブラ言語
字句解析・構文解析により、
プログラムの字面の構造を
解析する
複数の解析あるいは最適化を
順番に適用する
各パスの間の情報のやり取りは
中間表現(intermediate
representation)を介して行う
アセンブラ言語を
生成する
この後、アセンブラ・
リンカにより実行
ファイルとなる



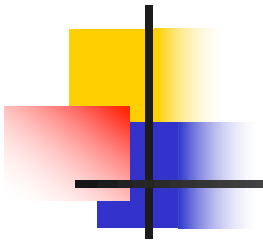
シンボルテーブル

- シンボルテーブル
  - プログラム中の変数等を管理するテーブル
- アルゴリズムとデータ構造の応用



シンボルテーブルで管理する情報

- 名前（変数、関数）
  - 型情報 (int, double等)
  - 変数に付随する情報
    - static, auto, const, サイズ等
  - スタック上の位置など
- 型
  - 配列
  - 構造体
  - ポインタ（何に対するポインタか）
  - 関数型（どのようなシグネチャか）
- 記憶域
  - メモリの使用状況など
- その他
  - ラベルなど
    - ラベルに関しては次回



中間表現

- コンパイラ内部でのデータ構造
- 用途に応じていくつかの種類がある
  - 字句解析・構文解析の結果を表現
  - アセンブリに近いデータ構造

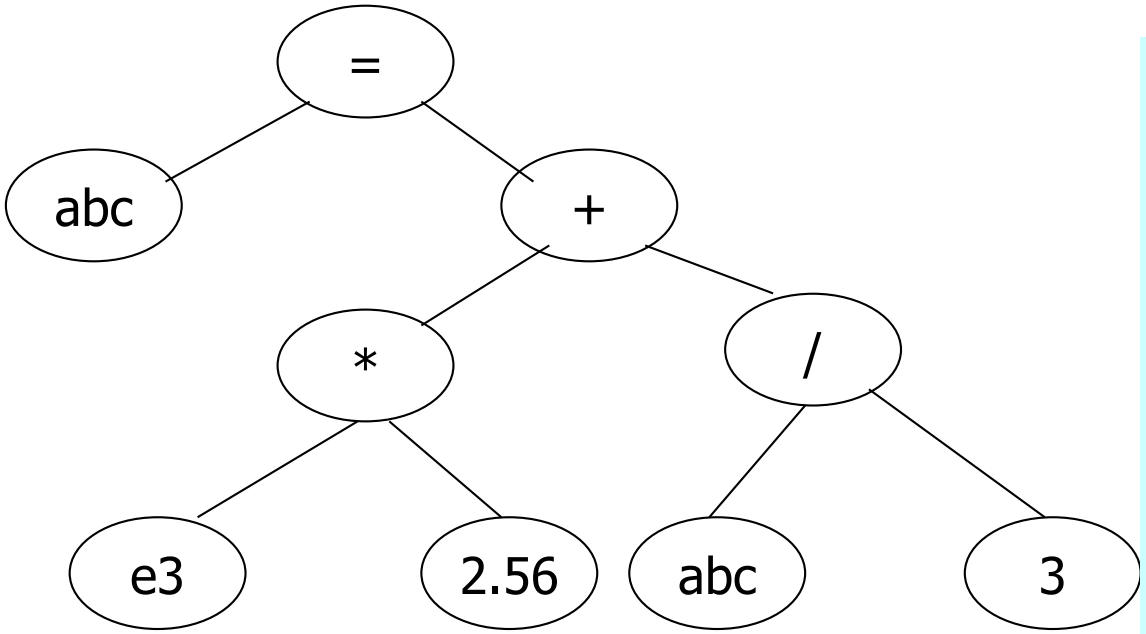
中間表現 (intermediate representation)

- コンパイラの各パス（フロントエンド、最適化器等）のインタフェース
  - 中間語 (intermediate language) という場合もある
- **構文木、抽象構文木 (abstract syntax tree)**
  - **プログラムの構造を示す木構造**
    - **解析木は具象構文木とも呼ぶ**
  - **構文解析の結果をそのまま反映する**
- 四つ組（3番地コード）
  - 構文木の構成要素を実行順に並べた列
  - 最適化に向く
- RTL (register transfer language)
  - アセンブリに近い形式
  - gccの中間表現
- LLVM-IR
  - LLVMの中間表現

本日扱う内容

構文木

- 例：abc=e3*2.56+abc/3;




```
struct AST_Node {
  int kind;
     // 関数/文/式
  int sub_kind;
     // 式の種類など
  struct AST_Node
    *child[N];
  union ASTType   val;
}
```

構文解析時に構築していく

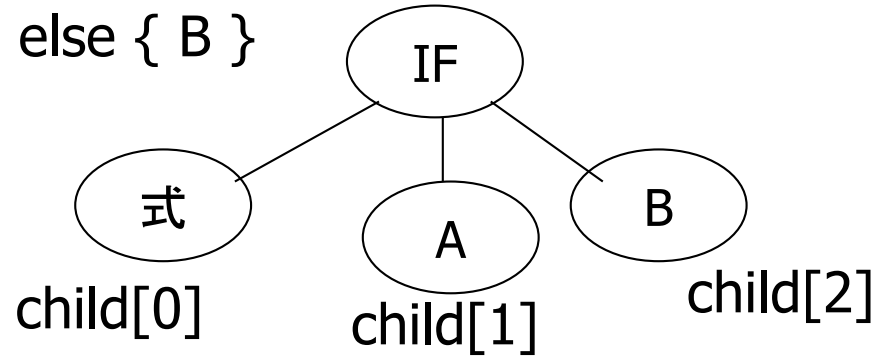
再帰的構文解析と構文木構築（第3回資料と比較）

```
AST_Node *E()
{
  AST_Node *n, *tn;
  n = T();
  while (token == SYM_PLUS) { // +
    tn = create_AST_Exp(AST_ADD);
    tn->child[0] = n;
    token = get_token();
    tn->child[1] = T();
    n = tn;
  }
  return n;
}
AST_Node *T()
{
  AST_Node *n, *fn;
  n = F();
  while (token == SYM_MULT) { // *
    fn = create_AST_Exp(AST_MUL);
    fn->child[0] = n;
    token = get_token();
    fn->child[1] = F();
    n = fn;
  }
  return n;
}
```

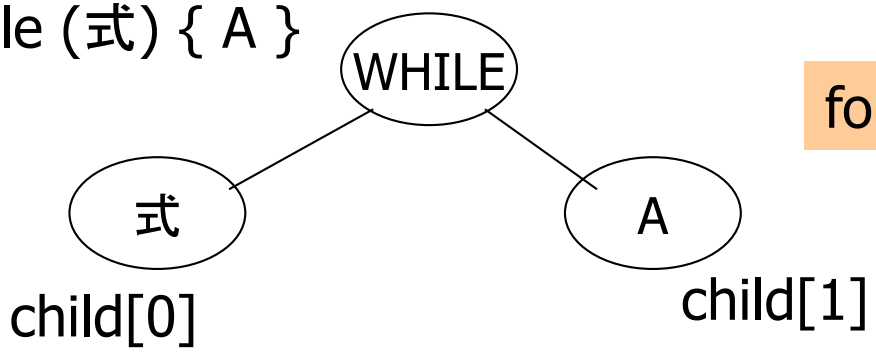
```
AST_Node *F()
{
  AST_Node *n;
  if (token == SYM_LEFTPAR) { // “(“
    token = get_token();
    n = E();
    if (token == SYM_RIGHTPAR) { // “)”
      token = get_token();
    } else {
      error();
    }
  } else if (token == <名前>) {
    n = create_AST_Exp(AST_IDENT);
    n->val.id = token.name;
    token = get_token();
  } else if (token == <数字>) {
    n = create_AST_Exp(AST_CNST);
    n->val.num = token.num;
    token = get_token();
  } else {
    error();
  }
  return n;
}
```

文の扱い(色々な方法がある)

if (式) { A } else { B }



while (式) { A }



forはwhileと等価

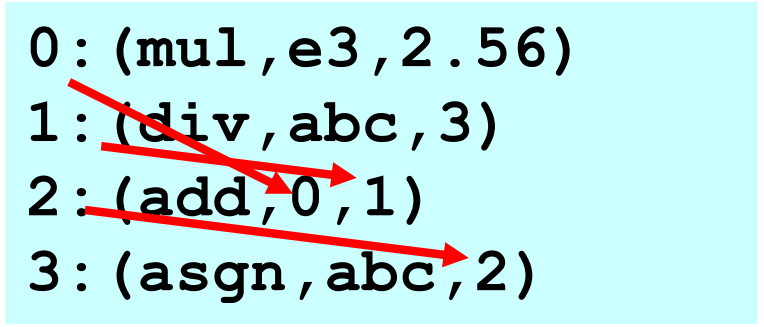
最終的にはifとgotoにより表現される(次回)

四つ組(3番地コード)と三つ組み

- 一行が一つの演算
  - 一つの式を表す抽象構文木中の各演算子を、演算子ごとに分割する
  - 上から辿ると適切な順番で演算できるようにする
  - 深さ優先探索で抽象構文木を辿り演算子を並べていく
- プログラム解析や最適化を行うのに便利
- 例:abc=e3*2.56+abc/3;

```
(mul,T1,e3,2.56)
(div,T2,abc,3)
(add,T3,T1,T2)
(asgn,abc,T3)
```

四つ組(3番地コード)

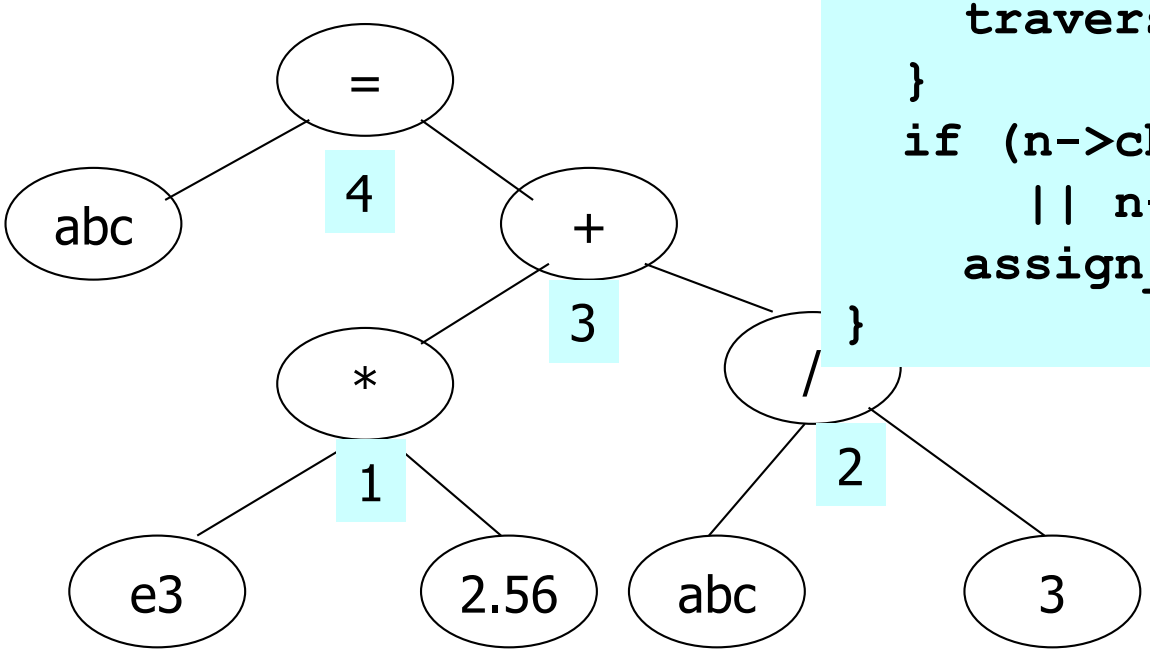


三つ組

(演算子,結果オペランド,ソースオペランド1,ソースオペランド2)

一時変数のつけ方

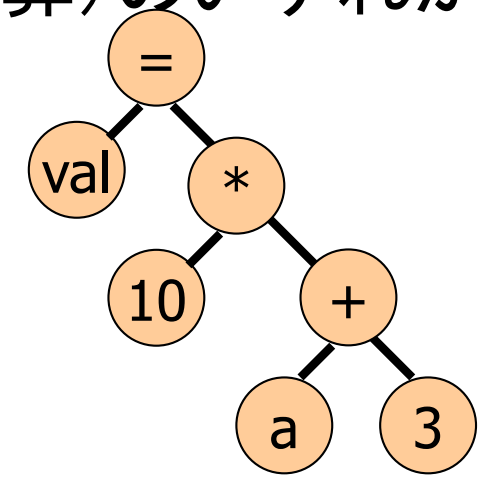
```
void traverse_node(AST_Node* n)
{
  if (n == NULL) {
    return;
  } else if (n->child[0] != NULL) {
    traverse_node(n->child[0]);
  } else if (n->child[1] != NULL) {
    traverse_node(n->child[1]);
  }
  if (n->child[0] != NULL
      || n->child[1] != NULL)
    assign_tmpvar(n);
}
```



深さ優先で一時変数をつけていく
（そのまま四つ組に変換できる）

演習問題(2)

- 以下の構文木に一時変数を割り当て、その後四つ組みを示せ。一時変数は先のスライドの通り深さ優先で番号を割り当てていくものとする。
- 四つ組みのフォーマットは(ope, dst, src1, src2)とする。ただし、dstは結果オペランド、src1, src2はそれぞれソースオペランドの1,2。また、opeはasgn（代入）, mul（乗算）, add（加算）のいずれか。



中間表現- gccの例

- gccの中間表現
  - GIMPLE
    - High/Low/SSA (Static Single Assignment)の各段階がある
    - -fdump-tree-all
  - RTL
    - 低レベルな中間表現
    - -fdump-rtl-all

サンプル

```
int nth;

int
main()
{
     if (nth > 0)
          nth += 10;
     return nth;
}
```

中間表現- GIMPLEとRTLの例

```
;; Function main (main)


main ()
{
  int D.1593;
  int nth.2;
  int nth.1;
  int nth.0;


<bb 2>:
  nth.0_1 = nth;
  if (nth.0_1 > 0)
    goto <bb 3>;
  else
    goto <bb 4>;


<bb 3>:
  nth.1_2 = nth;
  nth.2_3 = nth.1_2 + 10;
  nth = nth.2_3;


<bb 4>:
  D.1593_4 = nth;
  return D.1593_4;


}
```

```
;; Function main (main)

(略)

(insn 5 4 6 3 (set (reg:SI 59 [ nth.0 ])
        (mem/c/i:SI (symbol_ref:DI ("nth")  <var_decl 0x7f79eea34000 nth>)
[0 nth+0 S4 A32])) test.c:6 64 {*movsi_internal}
     (nil))


(insn 6 5 7 3 (set (reg:CCNO 17 flags)
        (compare:CCNO (reg:SI 59 [ nth.0 ])
            (const_int 0 [0]))) test.c:6 2 {*cmpsi_ccno_1}
     (nil))


(jump_insn 7 6 8 3 (set (pc)
        (if_then_else (le (reg:CCNO 17 flags)
                (const_int 0 [0]))
            (label_ref 12)
            (pc))) test.c:6 591 {*jcc_1}
     (nil)
 -> 12)


(note 8 7 9 4 [bb 4] NOTE_INSN_BASIC_BLOCK)


(insn 9 8 10 4 (set (reg:SI 60 [ nth.1 ])
        (mem/c/i:SI (symbol_ref:DI ("nth")  <var_decl 0x7f79eea34000 nth>)
[0 nth+0 S4 A32])) test.c:7 64 {*movsi_internal}
     (nil))
```

中間表現- LLVM-IR

- LLVMの中間表現
- SSA (static single assignment)をベースとする

```
int nth;

int
main()
{
    if (nth > 0)
        nth += 10;
    return nth;
}
```

```
@nth = common global i32 0, align 4

; Function Attrs: norecurse nounwind ssp uwtable
define i32 @main() #0 {
  %1 = load i32, i32* @nth, align 4, !tbaa !2
  %2 = icmp sgt i32 %1, 0
  br i1 %2, label %3, label %5

; <label>:3                                       ; preds = %0
  %4 = add nsw i32 %1, 10
  store i32 %4, i32* @nth, align 4, !tbaa !2
  br label %5

; <label>:5                                       ; preds = %3, %0
  %6 = phi i32 [ %4, %3 ], [ %1, %0 ]
  ret i32 %6
}
```

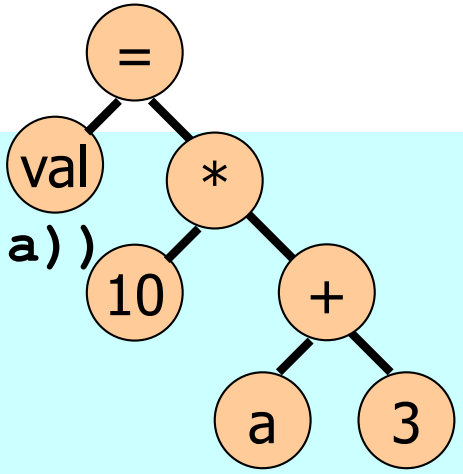
中間表現の例: ソースプログラムから構文木

```
int
foo ( int a )
{
    int val ;

    if ( a > 0 ) {
        val = 10 * ( a + 3 )
    } else {
        val = 10 * a + 3 ;
    }

    return  val ;
}
```

```
(function_table
  ("foo" int(args (int a))
(function "foo")
 (var_table
  (int val))
 (body
  (if ( GE a 0)
    ((ASGN val (MUL 10 ( ADD a 3))))
    ((ASGN val (ADD (MUL 10 a ) 3))))
  (return val)
))
```



構文解析の結果を反映している

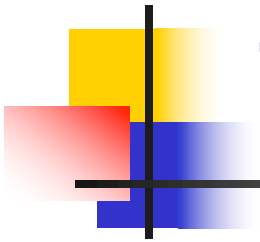
中間表現の例: 構文木から四つ組

```
(function_table
  ("foo" int(args (int a))
(function "foo")
 (var_table
  (int val))
 (body
  (if ( GE a 0)
    ((ASGN val (MUL 10 ( ADD a 3))))
    ((ASGN val (ADD (MUL 10 a ) 3))))
  (return val)
 ))
```

- 単純な処理を並べた形式
- 比較的アセンブラに近い
- 構造化文の扱いは次回に
  - ifとかwhileとか

一時変数

```
# body部分のみ
 (
  (ge,t1,a,0)
  (jf,t1,L1)
  (add,t2,a,3)
  (mul,t3,10,t2)
  (asgn,val,t3)
  (jmp,L2)
  (label,L1)
  (mul,t4,10,a)
  (add,t5,t4,3)
  (asgn,val,t5)
  (label,L2)
  (return,val)
 )
```



まとめ

- LR構文解析
  - LR(1), LALR(1)
- bisonによるサンプル
- エラー処理に関しても勉強してみよう
- シンボルテーブルといくつかの中間表現